WIREGUARD
FAST, MODERN, SECURE VPN TUNNEL

Sommaire

1. **Introduction**
2. **Contexte et objectifs de la mission**
3. **Topologie réseau et présentation des sites**
4. **Mise en place du routage inter-sites**
    - Activation de l'IP forwarding
    - Configuration des tables de routage
    - Tests de connectivité
5. **Déploiement d'un service FTP et analyse de sécurité**
    - Installation et configuration de vsftpd
    - Connexion FTP sans VPN
    - Capture et analyse des trames FTP
6. **Déploiement du VPN WireGuard**
    - Génération des clés
    - Configuration du site A
    - Configuration du site B
    - Mise en service du tunnel VPN
    - Tests de connectivité via VPN
7. **Analyse du trafic après mise en place du VPN**
    - Capture sur interface WAN
    - Capture sur interface wg0
    - Vérification du chiffrement
8. **Justification du choix de WireGuard**
9. **Conclusion**

Pourquoi avoir choisi WireGuard ?

Voici une justification claire, professionnelle et parfaitement adaptée à ton rapport :

✔ 1. Simplicité de configuration

WireGuard utilise un fichier de configuration minimaliste, avec seulement quelques paramètres essentiels (clé privée, clé publique, AllowedIPs, Endpoint). Cela le rend **beaucoup plus simple** à déployer que des solutions comme OpenVPN ou IPsec.

✔ 2. Performances supérieures

WireGuard repose sur des algorithmes modernes (ChaCha20, Poly1305, Curve25519) optimisés pour la vitesse. Résultat :

- Débit plus élevé
- Latence plus faible
- Consommation CPU réduite

Idéal pour un environnement pédagogique ou professionnel.

✔ 3. Sécurité moderne et robuste

Contrairement à IPsec ou OpenVPN, WireGuard n'utilise **que des primitives cryptographiques récentes**, sans rétrocompatibilité dangereuse. Cela réduit fortement les risques de mauvaise configuration.

✔ 4. Code source très léger

WireGuard fait environ **4 000 lignes de code**, contre plus de 100 000 pour OpenVPN. Un code plus court =

- moins de bugs
- moins de failles potentielles
- audit de sécurité plus facile

✔ 5. Intégration native dans Linux

Depuis Linux 5.6, WireGuard est intégré directement dans le noyau. Cela garantit :

- stabilité
- performance
- maintenance à long terme

✅ 6. Parfait pour un tunnel site-à-site

Grâce aux AllowedIPs, WireGuard agit comme un routeur chiffré. Il permet de transporter facilement un LAN entier dans un tunnel sécurisé.

➡️ **En résumé : WireGuard est moderne, rapide, simple, sécurisé et parfaitement adapté à un VPN site-à-site.**

1. Introduction

Dans le cadre de la mission confiée par l'entreprise Egnom pour le client Selenia Software, il s'agissait de déployer un VPN permettant d'interconnecter deux sites distants. Chaque machine virtuelle représente un site, et le réseau 10.10.0.0/16 simule Internet.

Les objectifs étaient les suivants :

- Mettre en place le routage entre les deux sites.
- Installer un service simple (FTP) et démontrer son manque de sécurité via une capture de trames.
- Déployer une solution VPN (WireGuard).
- Vérifier que les communications sont désormais chiffrées.

Ce compte rendu présente l'ensemble des étapes réalisées, les configurations mises en place et les résultats obtenus.

2. Contexte et topologie réseau

Deux sites ont été configurés :

Site A (debian)

```
admin1@debian: ~
root@debian:/# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:d0:75:1e brd ff:ff:ff:ff:ff:ff
    altname enx00155dd0751e
    inet 172.30.17.1/24 brd 172.30.17.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::444b:8afb:8bb1:2eab/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:d0:75:28 brd ff:ff:ff:ff:ff:ff
    altname enx00155dd07528
    inet 10.10.17.10/16 scope global eth1
       valid_lft forever preferred_lft forever
5: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN group default qlen 1000
    link/none
    inet 10.0.0.1/24 scope global wg0
       valid_lft forever preferred_lft forever
```

Site B (admin1)

```
root@admin1-Virtual-Machine:/# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:d0:75:27 brd ff:ff:ff:ff:ff:ff
    inet 172.30.17.30/24 brd 172.30.17.255 scope global noprefixroute eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::10f8:94f:f74a:64eb/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN group default qle
n 1000
    link/none
    inet 10.0.0.2/24 scope global wg0
       valid_lft forever preferred_lft forever
root@admin1-Virtual-Machine:/#
```

Le réseau 10.10.0.0/16 représente Internet et permet la communication entre les deux sites.

3. Mise en place du routage inter-sites

3.1 Activation de l'IP forwarding

Sur les deux machines :

Site A

```
root@debian:/# cat /proc/sys/net/ipv4/ip_forward
sysctl net.ipv4.ip_forward
1
net.ipv4.ip_forward = 1
```

Site B

```
root@admin1-Virtual-Machine:/# cat /proc/sys/net/ipv4/ip_forward
sysctl net.ipv4.ip_forward
1
net.ipv4.ip_forward = 1
```

Cela permet au système de router les paquets entre interfaces.

3.2 Table de routage du site A

```
root@debian:/# ip route show
default via 172.30.17.254 dev eth0 proto static metric 100
10.0.0.0/24 dev wg0 proto kernel scope link src 10.0.0.1
10.10.0.0/16 dev eth1 proto kernel scope link src 10.10.17.10
172.30.17.0/24 dev eth0 proto kernel scope link src 172.30.17.1 metric 100
172.30.17.30 dev wg0 scope link
root@debian:/#
```

Routes observées :

- Route par défaut via 172.30.17.254
- Route vers le LAN B via eth1
- Route vers le réseau VPN via wg0

3.3 Table de routage du site B

```
root@admin1-Virtual-Machine:~# ip route show
default via 172.30.17.254 dev eth0 proto static metric 100
10.0.0.0/24 dev wg0 proto kernel scope link src 10.0.0.2
172.30.17.0/24 dev wg0 scope link
172.30.17.0/24 dev eth0 proto kernel scope link src 172.30.17.30 metric 100
```

Routes observées :

- Route par défaut via 172.30.17.254
- Route vers le réseau VPN via wg0
- Route vers le LAN A via wg0

3.4 Tests de connectivité

- Ping entre les deux LAN : OK
- Traceroute : un seul saut via le tunnel VPN

Le routage est fonctionnel.

4. Mise en place d'un service FTP et analyse de sécurité

4.1 Installation du service FTP

```
root@debian:/# sudo systemctl status vsftpd
● vsftpd.service - vsftpd FTP server
     Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-12-18 15:03:56 CET; 12min ago
 Invocation: a3f95cb947bb41858743d90a8431ca9c
   Main PID: 7213 (vsftpd)
      Tasks: 1 (limit: 4602)
     Memory: 960K (peak: 18.8M)
        CPU: 151ms
     CGroup: /system.slice/vsftpd.service
             └─7213 /usr/sbin/vsftpd /etc/vsftpd.conf

déc. 18 15:03:56 debian systemd[1]: Starting vsftpd.service - vsftpd FTP server...
déc. 18 15:03:56 debian systemd[1]: Started vsftpd.service - vsftpd FTP server.
```

Le service **vsftpd** a été installé et lancé correctement.

4.2 Connexion FTP sans VPN

```
root@admin1-Virtual-Machine:/# ftp 172.30.17.1
Connected to 172.30.17.1.
220 (vsFTPd 3.0.5)
Name (172.30.17.1:admin1): testftp
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||55161|)
150 Here comes the directory listing.
226 Directory send OK.
ftp> quit
221 Goodbye.
```

Depuis le site B :

- Authentification réussie
- Listing de fichiers OK

4.3 Capture de trames FTP sans VPN

Commande utilisée :

```
root@debian:/# sudo tcpdump -i eth0 -w /tmp/ftp_sans_vpn.pcap port 21 -v
tcpdump: /tmp/ftp_sans_vpn.pcap: Permission denied
root@debian:/# ls -l /tmp/ftp_sans_vpn.pcap
-rw-r--r-- 1 tcpdump tcpdump 1740 18 déc.  15:07 /tmp/ftp_sans_vpn.pcap
root@debian:/# sudo rm -f /tmp/ftp_sans_vpn.pcap
root@debian:/# sudo tcpdump -i eth0 port 21 -w /tmp/ftp_sans_vpn.pcap -v
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C17 packets captured
17 packets received by filter
0 packets dropped by kernel
root@debian:/# sudo tcpdump -r /tmp/ftp_sans_vpn.pcap -A | grep -E "USER|PASS"
reading from file /tmp/ftp_sans_vpn.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:23:11.700325 IP 172.30.17.30.48664 > debian.ftp: Flags [P.], seq 0:14, ack 21, win 32480, options [nop,nop,TS
 val 1821275067 ecr 193604824], length 14: FTP: USER testftp
l.s...,.USER testftp
15:23:16.411617 IP 172.30.17.30.48664 > debian.ftp: Flags [P.], seq 14:32, ack 55, win 32472, options [nop,nop,T
S val 1821279779 ecr 193610764], length 18: FTP: PASS Password123
l..#..D.PASS Password123
root@debian:/#
```

Analyse :

- Le login apparaît en clair : `USER testftp`
- Le mot de passe apparaît en clair : `PASS Password123`

Démonstration claire du manque de sécurité du protocole FTP.

5. Déploiement du VPN WireGuard

5.1 Génération des clés

Sur chaque site :

- Une clé privée
- Une clé publique

Site A

```
root@debian:/# cd /etc/wireguard
sudo cat publickey
sudo cat privatekey
98yae80yw/0eZRXyu/pgglWnEryWutJw3JFgy6ykA1A=
sO6vYkH/Fk0nHkpIwDWv+k5KeTq7u3/fSeBVfSIz/0M=
```

Site B

```
root@admin1-Virtual-Machine:/# cd /etc/wireguard
sudo cat publickey
sudo cat privatekey
8N1VQjEaa04wzlglG4PAdt/rbf0l2rl6D+ENywmY3GM=
4H3utJZZoxxPSVpJdypFVFLlBhY1oVsziZjNqPwJxWI=
root@admin1-Virtual-Machine:/etc/wireguard#
```

5.2 Configuration du site A

```
root@debian:/etc/wireguard# sudo cat /etc/wireguard/wg0.conf
[Interface]
Address = 10.0.0.1/24
PrivateKey = SHIwVTC9hoXf8ugQSYgdm8z6MgELpEq9fn0ho9iuR0Q=
ListenPort = 51820

[Peer]
PublicKey = 8N1VQjEaa04wzlglG4PAdt/rbf0l2rl6D+ENywmY3GM=
Endpoint = 10.10.17.20:51820
AllowedIPs = 10.0.0.2/32, 172.30.17.30/32
PersistentKeepalive = 25
```

5.3 Configuration du site B

```
root@admin1-Virtual-Machine:/etc/wireguard# sudo cat /etc/wireguard/wg0.conf
[Interface]
Address = 10.0.0.2/24
PrivateKey = 4H3utJZZoxxPSVpJdypFVFLlBhY1oVsziZjNqPwJxWI=
ListenPort = 51820

[Peer]
PublicKey = g5klMvR0n870Sz6H/V1oHhqGgBr9DgJ7W3WX+fX5eQo=
Endpoint = 10.10.17.10:51820
AllowedIPs = 10.0.0.1/32, 172.30.17.0/24
PersistentKeepalive = 25
```

5.4 Mise en service du VPN

Site A

```
root@debian:/etc/wireguard# sudo wg-quick up wg0
sudo wg show
wg-quick: `wg0' already exists
interface: wg0
  public key: g5klMvR0n870Sz6H/V1oHhqGgBr9DgJ7W3WX+fX5eQo=
  private key: (hidden)
  listening port: 51820

peer: 8N1VQjEaa04wzlglG4PAdt/rbf0l2rl6D+ENywmY3GM=
  endpoint: 10.10.0.3:51820
  allowed ips: 10.0.0.2/32, 172.30.17.30/32
  latest handshake: 1 minute, 22 seconds ago
  transfer: 9.54 KiB received, 17.00 KiB sent
  persistent keepalive: every 25 seconds
```

Site B

```
root@admin1-Virtual-Machine:/etc/wireguard# sudo wg show
interface: wg0
  public key: 8N1VQjEaa04wzlglG4PAdt/rbf0l2rl6D+ENywmY3GM=
  private key: (hidden)
  listening port: 51820

peer: g5klMvR0n870Sz6H/V1oHhqGgBr9DgJ7W3WX+fX5eQo=
  endpoint: 10.10.17.10:51820
  allowed ips: 10.0.0.1/32, 172.30.17.0/24
  latest handshake: 8 seconds ago
  transfer: 13.72 KiB received, 9.77 KiB sent
  persistent keepalive: every 25 seconds
```

Vérification :

- Handshake OK
- Transferts de données visibles
- Endpoint actif
- Keepalive fonctionnel

5.5 Tests de connectivité via VPN

- Ping 10.0.0.2 → OK
- Ping 172.30.17.30 → OK
- Traceroute → passage par wg0

Le tunnel VPN est opérationnel.

6. Analyse du trafic après mise en place du VPN

6.1 Capture sur interface WAN

Commande :

```
root@debian:/etc/wireguard# sudo tcpdump -i eth1 port 51820 -w /tmp/ftp_avec_vpn_wan.pcap -c 50 -v
tcpdump: listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C40 packets captured
40 packets received by filter
0 packets dropped by kernel
root@debian:/etc/wireguard# sudo tcpdump -r /tmp/ftp_avec_vpn_wan.pcap -X
reading from file /tmp/ftp_avec_vpn_wan.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:36:38.753453 IP 10.10.0.3.51820 > debian.51820: UDP, length 96
        0x0000:  4500 007c 9041 0000 3f11 c60f 0a0a 0003  E..|.A..?.......
        0x0010:  0a0a 110a ca6c ca6c 0068 fb14 0400 0000  .....l.l.h......
        0x0020:  9bfe 2d72 1800 0000 0000 0000 9547 ada9  ..-r.........G..
        0x0030:  bd16 379a 536b bf60 a538 da57 ec75 2610  ..7.Sk.`.8.W.u&.
        0x0040:  90a2 3fc6 3656 6d4a ee8c b38f 44e3 cada  ..?.6VmJ....D...
        0x0050:  f9b0 cab7 3891 a310 602d e7cb 50fe e1ed  ....8...`-..P...
        0x0060:  a940 91eb 55ac 8739 d31d 9f82 fa30 e884  .@..U..9.....0..
        0x0070:  894d e4d0 d779 fe14 229d 7392            .M...y..".s.
15:36:38.753708 IP debian.51820 > 10.10.0.3.51820: UDP, length 96
        0x0000:  4500 007c c3e1 0000 4011 916f 0a0a 110a  E..|....@..o....
        0x0010:  0a0a 0003 ca6c ca6c 0068 259a 0400 0000  .....l.l.h%.....
        0x0020:  73c9 2758 1200 0000 0000 0000 54d6 f560  s.'X........T..`
        0x0030:  5cb8 d992 68b7 20b5 8c87 4ddc e77a 78d8  \...h.....M..zx.
        0x0040:  6471 0670 a983 5bf1 3134 7f37 b087 44f5  dq.p..[.14.7..D.
        0x0050:  9474 3f56 2b15 7086 384a d4c8 46ad 2c47  .t?V+.p.8J..F.,G
        0x0060:  f41a b5cf 96f2 88fd 2627 61cc 2fac ea0a  ........&'a./...
        0x0070:  4bf3 9721 b0c8 0864 37f7 cf15            K..!...d7...
15:36:38.754382 IP 10.10.0.3.51820 > debian.51820: UDP, length 96
        0x0000:  4500 007c 9042 0000 3f11 c60e 0a0a 0003  E..|.B..?.......
        0x0010:  0a0a 110a ca6c ca6c 0068 e8ef 0400 0000  .....l.l.h......
        0x0020:  9bfe 2d72 1900 0000 0000 0000 fdea cc14  ..-r............
        0x0030:  42dd 06f1 0919 d41f 8816 2c42 4bdb 0604  B.........,BK...
        0x0040:  4005 f818 633e b96d fa7f 103c 1fae b462  @...c>.m...<...b
        0x0050:  7dc2 6bf1 e0e1 f008 c0d0 ef8b da52 af84  }.k..........R..
        0x0060:  d411 56be e025 bcd9 024d 43f1 d391 07cf  ..V..%...MC.....
        0x0070:  96cb ca63 abd1 1f8f b7bb 885f            ...c........_
15:36:38.755625 IP debian.51820 > 10.10.0.3.51820: UDP, length 112
        0x0000:  4500 008c c3e2 0000 4011 915e 0a0a 110a  E.......@..^....
        0x0010:  0a0a 0003 ca6c ca6c 0078 25aa 0400 0000  .....l.l.x%.....
        0x0020:  73c9 2758 1300 0000 0000 0000 9126 e163  s.'X.........&.c
```

Analyse :

- Le trafic est encapsulé dans UDP/51820
- Les données sont chiffrées
- Impossible de lire les identifiants FTP

6.2 Capture sur interface wg0

Commande :

```
root@debian:/etc/wireguard# sudo tcpdump -i wg0 port 21 -w /tmp/ftp_tunnel.pcap -v
tcpdump: listening on wg0, link-type RAW (Raw IP), snapshot length 262144 bytes
^C34 packets captured
34 packets received by filter
0 packets dropped by kernel
root@debian:/etc/wireguard# sudo tcpdump -r /tmp/ftp_tunnel.pcap -A | grep -E "USER|PASS"
reading from file /tmp/ftp_tunnel.pcap, link-type RAW (Raw IP), snapshot length 262144
15:45:09.129344 IP 10.0.0.2.57896 > debian.ftp: Flags [P.], seq 1:15, ack 21, win 32426, options [nop,nop,TS val
 158625615 ecr 2352282928], length 14: FTP: USER testftp
        toO.4.0USER testftp
15:45:13.889751 IP 10.0.0.2.57896 > debian.ftp: Flags [P.], seq 15:33, ack 55, win 32426, options [nop,nop,TS va
l 158630375 ecr 2352287821], length 18: FTP: PASS Password123
        t...5.MPASS Password123
```

Analyse :

- Le trafic FTP est encapsulé dans WireGuard
- Les données sont chiffrées
- Aucune information sensible n'est visible

Le VPN sécurise totalement les échanges.

7. Justification du choix de WireGuard

WireGuard a été choisi pour plusieurs raisons :

- Simplicité de configuration

Quelques lignes suffisent pour établir un tunnel fonctionnel.

- Performances élevées

Grâce à ChaCha20 et Poly1305, WireGuard offre un excellent débit et une faible latence.

- Sécurité moderne

Pas de cryptographie obsolète, pas de rétrocompatibilité dangereuse.

- Code source léger

Environ 4 000 lignes, contre plus de 100 000 pour OpenVPN. Moins de bugs, moins de failles potentielles.

- Intégration native dans Linux

Depuis Linux 5.6, WireGuard est intégré au noyau, garantissant stabilité et performance.

- Idéal pour un VPN site-à-site

AllowedIPs permet de router facilement des réseaux entiers dans le tunnel.

WireGuard est moderne, rapide, sécurisé et parfaitement adapté à ce projet.

8. Conclusion

La mise en place du VPN entre les deux sites de Selenia Software a été réalisée avec succès. Après avoir configuré le routage inter-sites et démontré l’insécurité du protocole FTP via une capture de trames, un tunnel VPN WireGuard a été déployé. Les tests ont confirmé que les communications sont désormais chiffrées et protégées.

WireGuard s’est révélé être une solution particulièrement adaptée grâce à sa simplicité, ses performances et son haut niveau de sécurité. Les deux sites peuvent maintenant échanger des données de manière confidentielle, fiable et conforme aux attentes du client.

La mission est donc pleinement accomplie.